

ShipFlow Backend Roadmap

Superseded on 2026-08-11 by the [unified project roadmap](#). Retained as a historical team-specific planning view.

- Repository: `shipflow-backend`
- Team ownership: Backend team
- Baseline date: 2026-08-07
- Current branch policy: Work directly on `main`
- Current implementation baseline: Backend Phase 12 complete

Purpose

This document contains only backend-owned work for ShipFlow. It preserves the phase numbers from the original project plan so that cross-team references stay stable. Phase numbers that belong entirely to the frontend are intentionally omitted.

The backend owns database integrity, authentication, tenant isolation, authorization, provider integrations, API contracts, operational behavior, and backend deployment documentation. The frontend may improve usability around these capabilities, but it is never the security boundary.

Status Legend

Status	Meaning
Complete	Implemented, verified, documented, committed, and pushed
In progress	Some decisions or implementation remain
Decision gate	Product or commercial rules must be approved before code
Pending	Not started
Deferred	Deliberately moved to the phase that owns the requirement

Phase Summary

Original phase	Backend scope	Status	Notes
Phase 0	Architecture and backend product decisions	In progress	Deployment, Redis, storage, and release-policy decisions remain
Phase 1	Backend repository foundation	Complete	Toolchain, scripts, environment template, and repository baseline established
Phase 2	Local backend infrastructure	Complete	PostgreSQL 18.4 Compose service verified
Phase 3	NestJS backend foundation	Complete	API conventions, Prisma, health, validation, errors, CORS, and Swagger established
Phase 4	Initial identity and tenancy schema	Complete	Billing, notification, and file models deferred to their owning phases
Phase 5	Backend authentication	Complete	Production email delivery integrated in Phase 9
Phase 7	Organizations and multi-tenancy	Complete	Tenant lifecycle, invitations, membership management, and isolation verified
Phase 8	Role-based access control	Complete	Capability matrix, guards, request context, and exhaustive role coverage
Phase 9	Transactional email	Complete	Shared delivery port, Resend adapter, templates, metadata, and webhooks
Phase 10	Stripe billing	Complete	Owner APIs, hosted Stripe flows, local entitlements, and webhook synchronization
Phase 12	Notification APIs and persistence	Complete	User-owned APIs, preferences, retention, and transactional domain events
Phase 13	File storage APIs and provider integration	Complete	Local development and private S3 production adapters implemented
Phase 14	Backend production hardening	In progress	Provider-independent single-instance baseline implemented
Phase 15	Backend CI/CD, regression testing, and documentation	Pending	Final repeatability and buyer-readiness work
Release	Backend release support	Pending	Supports beta, packaging, upgrades, and commercial launch

Phase 0: Backend Architecture and Product Decisions

Status: In progress

Objective: Remove ambiguity before security-sensitive, tenant-sensitive, or provider-sensitive backend work begins.

Completed Decisions

1. Use a polyrepo architecture with independently deployable backend and frontend repositories.
2. Use NestJS 11, Node.js 24, pnpm 11, PostgreSQL, Prisma, and a versioned REST API.
3. Use PostgreSQL 18.4 locally with native UUID v7 identifiers.
4. Use normalized lowercase email addresses and UTC-aware timestamps.
5. Use 15-minute access tokens and 30-day rotating refresh sessions.
6. Store refresh tokens in HttpOnly cookies and only their SHA-256 hashes in PostgreSQL.
7. Allow concurrent sessions and use rotation-family reuse detection.
8. Allow users without organizations during onboarding.
9. Use one explicit organization owner plus Owner, Admin, Member, and Viewer memberships.
10. Use seven-day, single-use organization invitations.
11. Keep Redis out of the current runtime until it has an approved responsibility.
12. Use Swagger/OpenAPI as the initial backend-to-frontend contract.
13. Use the ADR 0006 Owner, Admin, Member, and Viewer capability matrix.

Remaining Decisions

1. Decide whether Redis will support distributed rate limits, queues, caching, or another explicit production responsibility.
2. Select the primary backend deployment platform and production PostgreSQL provider.
3. Define API compatibility, generated-client, and coordinated release policies with the frontend team before Phase 15.

Completion Criteria

1. Every remaining decision has an accepted ADR or an explicit deferral.
2. No implementation phase needs to invent security or commercial product rules.

Phase 1: Backend Repository Foundation

Status: Complete

Objective: Provide an independently maintainable backend repository with a repeatable toolchain and quality baseline.

Completed Steps

1. Created and pushed the backend Git repository.
2. Scaffolded NestJS with strict TypeScript settings.
3. Pinned Node.js 24 and pnpm 11 expectations.
4. Added install, format, lint, test, e2e, build, Prisma, and development scripts.
5. Added environment templates without committing local secrets.
6. Added LF line-ending policy for consistent Windows and CI formatting.
7. Established conventional commit messages and direct-to-main workflow for the current team.

Completion Criteria

1. A clean checkout can install, validate, test, and build independently.
2. No local `.env` file or credential is tracked.

Phase 2: Local Backend Infrastructure

Status: Complete

Objective: Provide a repeatable local PostgreSQL runtime for backend work.

Completed Steps

1. Added PostgreSQL 18.4 through Docker Compose.
2. Bound PostgreSQL to the loopback interface with development-only credentials.
3. Added health checks, a persistent named volume, checksums, and a backend network.
4. Verified startup, readiness, persistence, restart, backup, and restore behavior.
5. Documented reset and recovery commands.
6. Deferred Redis until an explicit production purpose is approved.

Completion Criteria

1. PostgreSQL starts consistently and reports healthy.
2. The backend connects successfully and data survives container recreation.

Phase 3: NestJS Backend Foundation

Status: Complete

Objective: Establish reusable application behavior shared by every backend feature module.

Completed Steps

1. Added global validated configuration through `ConfigModule` and Joi.
2. Added `/api/v1` prefixing and URI versioning.
3. Added strict global DTO validation with allowlisting and unknown-property rejection.
4. Added consistent API error responses and request IDs.
5. Added request logging and graceful shutdown.
6. Added validated CORS allowlisting with credentials support.
7. Added Swagger UI and OpenAPI JSON generation.
8. Added Prisma 7 with the PostgreSQL driver adapter and a global `PrismaService`.
9. Added liveness and database-backed readiness endpoints.
10. Added focused unit and e2e coverage for startup, validation, health, and API conventions.

Completion Criteria

1. `/api/v1/health/ready` confirms configuration and database availability.
2. Swagger exposes the versioned API contract.
3. Format, lint, tests, e2e tests, and build pass.

Phase 4: Initial Identity and Tenancy Schema

Status: Complete

Objective: Establish the relational baseline required by authentication and organizations without prematurely defining later provider models.

Completed Steps

1. Added User, OAuthAccount, Session, EmailVerificationToken, PasswordResetToken, Organization, Membership, and Invitation models.
2. Added UUID v7 identifiers and mapped Prisma names to snake_case PostgreSQL names.
3. Added native UUID, inet, and timestamptz PostgreSQL types.
4. Added normalized-email checks, unique constraints, foreign keys, indexes, and lifecycle constraints.
5. Enforced one Owner membership and one pending invitation per organization and email.
6. Added refresh-token family and replacement tracking.
7. Added initial and corrective migrations.
8. Added an idempotent development seed with two users, one organization, and memberships.
9. Added live database contract tests and schema ADRs.

Explicit Deferrals

1. Plan, subscription, Stripe customer, and webhook-event models move to Phase 10 after billing rules are approved.
2. Notification models move to Phase 12 after notification semantics are approved.
3. File metadata models move to Phase 13 after storage and upload policies are approved.

Completion Criteria

1. Migrations apply cleanly and the seed is idempotent.
2. The schema supports authentication and organization flows.
3. Database constraints are protected by automated tests.

Phase 5: Backend Authentication

Status: Complete

Objective: Deliver a secure, tested backend account lifecycle.

Completed Steps

1. Added normalized registration with duplicate-email handling.
2. Added Argon2id password hashing and nullable password hashes for OAuth-only users.
3. Added local login with safe invalid-credential responses.
4. Added HS256 access tokens and a database-backed access guard.
5. Added opaque refresh cookies, atomic rotation, family tracking, and replay revocation.
6. Added current-session logout, logout-all, and current-user endpoints.
7. Added single-use email-verification and password-reset flows.
8. Added password-reset session revocation.
9. Added cookie-origin checks and endpoint-specific in-memory rate limits.
10. Added optional Google and GitHub OAuth with verified-email requirements and signed state cookies.
11. Added a replaceable auth-delivery interface and development link logging.
12. Added Swagger contracts, security ADRs, unit tests, and a full database-backed e2e lifecycle.
13. Added a scoped dependency override resolving the Swagger `js-yaml` advisory.

Configuration Still Required Outside Code

1. Supply Google credentials to enable Google OAuth.
2. Supply GitHub credentials to enable GitHub OAuth.
3. Configure production Resend credentials, sender DNS, and webhook endpoint.

Completion Criteria

1. Registration, login, refresh, replay detection, logout, verification, reset, and OAuth mapping are tested.
2. Passwords and raw persistent tokens never appear in database records or API responses.
3. Production startup requires a strong explicit JWT secret and secure cookies.

Phase 7: Organizations and Multi-Tenancy

Status: Complete

Objective: Establish organizations as the tenant boundary and prove that cross-tenant access is impossible through supported APIs.

Accepted Implementation Baseline

1. Use explicit organization IDs in route paths instead of hidden global tenant state.
2. Require verified email before organization creation or invitation acceptance.
3. Create an organization and its Owner membership in one transaction.
4. Generate normalized unique slugs from organization names with collision suffixes.
5. Require invitation acceptance by an authenticated user with the matching normalized email.
6. Let Owner and Admin manage invitations and ordinary memberships during Phase 7.
7. Reserve ownership transfer and organization deletion for the Owner.
8. Require ownership transfer before the Owner can leave.
9. Keep soft-deleted organizations inaccessible and defer restore and purge workflows.

Completed Steps

1. Defined organization, invitation, membership, and ownership-transfer DTOs and response contracts.
2. Implemented organization creation, listing, retrieval, updating, and soft deletion.
3. Added reusable organization-context resolution based on route parameters.
4. Added membership lookup helpers that always scope by both user and organization.
5. Implemented invitation creation, listing, resend, revocation, expiry, and acceptance.
6. Implemented member listing, role changes, removal, and voluntary departure.
7. Implemented transactional ownership transfer and prevented invalid Owner removal.
8. Added page-based pagination and stable error semantics for organization collections.
9. Added negative cross-tenant tests for every tenant-scoped operation.
10. Extended Swagger and recorded the organization lifecycle in ADR 0005.

Satisfied Dependencies

1. Phase 4 organization schema.
2. Phase 5 authenticated-user context.
3. Approval of the Phase 7 baseline above.

Completion Criteria

1. Users can belong to multiple organizations and list only their memberships.
2. Invitations are safe, expiring, single-use, and email-bound.
3. Ownership transfer preserves exactly one Owner.
4. Cross-tenant requests fail even when valid resource IDs are supplied.

Phase 8: Role-Based Access Control

Status: Complete

Objective: Apply the final Owner, Admin, Member, and Viewer permission matrix consistently across backend modules.

Completed Steps

1. Approved a capability matrix covering organization, membership, invitation, billing, notification, file, and destructive actions.
2. Represented permissions as capabilities instead of scattered role comparisons.
3. Added reusable membership and capability decorators and an organization authorization guard.
4. Added a central authorization service for role and resource-sensitive decisions.
5. Protected every organization-scoped mutation and sensitive read.
6. Added table-driven allow and deny tests for every role and capability.
7. Documented how later modules register and enforce new capabilities in ADR 0006.

Completion Criteria

1. Every restricted action is enforced by the backend.
2. Every role has automated positive and negative authorization coverage.
3. Feature modules do not contain ad hoc, contradictory role logic.

Phase 9: Transactional Email

Status: Complete

Objective: Provide replaceable, observable delivery for authentication and organization messages.

Ordered Steps

1. Promote the existing auth-delivery boundary into a general email port.
2. Select and configure Resend as the reference adapter.
3. Keep a local log or capture adapter for development and tests.
4. Add verification, password-reset, invitation, and security-notice templates.
5. Validate sender, reply-to, support, frontend URL, and provider credentials.
6. Define retry policy, provider error translation, logging, and failure behavior.
7. Prevent secrets and full sensitive tokens from entering production logs.
8. Add contract tests that do not call the live provider.
9. Add one controlled provider smoke-test procedure for deployment validation.

Delivered

1. Replaced the auth and organization delivery adapters with one provider-neutral transactional email interface.
2. Added Resend, local log, and test capture adapters with bounded retries and idempotency keys.
3. Added source-controlled HTML and plain-text verification, reset, invitation, and security-notice templates.
4. Added delivery metadata persistence without raw tokens, rendered content, or complete webhook payloads.
5. Added raw-body Resend signature verification, event deduplication, and out-of-order status protection.
6. Added provider-free contract tests, database-backed failure-boundary tests, and a controlled smoke command.

Completion Criteria

1. Core account and organization flows send through an application interface.
2. Provider SDK types do not leak into business services.
3. Tests run without a live email account.

Phase 10: Stripe Billing

Status: Complete

Objective: Add organization-level subscription and payment management with reliable local entitlements.

Accepted Billing Rules

1. Use Free plus one organization-level Pro product at USD 29 monthly or USD 290 annually.
2. Offer one card-required 14-day trial per organization and a seven-day failed-payment grace period.
3. Use period-end cancellation and Portal reactivation; reserve immediate cancellation for full refunds, disputes, fraud, and administration.
4. Use Stripe Smart Retries, Stripe failed-payment emails, manual refunds, and Stripe Tax with exclusive USD prices.
5. Keep Stripe authoritative for payments and the local database authoritative for request-time entitlements.

Delivered

1. Added Plan, Subscription, StripeCustomer, BillingCheckoutSession, and ProcessedStripeEvent models, constraints, catalog seeds, and migrations.
2. Added official Stripe and local stub adapters behind an application billing interface.
3. Added one idempotently created Stripe customer, one pending Checkout reservation, and one subscription per organization.
4. Added Owner-only Checkout and Customer Portal session APIs plus public plan discovery.
5. Added card-required trials, billing-address and tax-ID collection, and production Stripe Tax enforcement.
6. Added raw-body Stripe signature verification and event deduplication.
7. Added current-state subscription retrieval, local transactions, and out-of-order timestamp protection.
8. Added local status, period, invoice, cancellation, trial, and failed-payment grace synchronization.
9. Added request-time entitlement checks with no Stripe dependency.
10. Added provider, entitlement, lifecycle, authorization, replay, ordering, refund, and failure regression tests.

Completion Criteria

1. Organization billing state remains synchronized with Stripe.
2. Duplicate and out-of-order webhook events are safe.
3. Backend authorization uses local verified entitlements.

Phase 12: Notification Backend

Status: Complete

Objective: Provide persistent, tenant-safe user notifications and preferences.

Accepted Defaults

1. Use persistent in-app Security, Organization, and Billing notifications without adding real-time infrastructure.
2. Make records user-owned with an optional organization reference and safe source-controlled content.
3. Keep security and billing mandatory; make organization activity optional and enabled by default.
4. Retain records for 90 days and hard-delete expired data.
5. Use newest-first opaque cursor pagination with a default of 20 and maximum of 100.
6. Support 30-second visibility-aware frontend polling, unread filters, and organization filters.

Delivered

1. Added Notification and NotificationPreference models, enums, constraints, indexes, and migration.
2. Added list, unread-count, mark-read, mark-all-read, and preference APIs.
3. Added opaque `(createdAt, id)` cursor pagination and stable newest-first ordering.
4. Added user isolation, optional organization filtering, and mandatory-category preference semantics.
5. Added transactional password, invitation, role, removal, ownership, trial, payment, grace, cancellation, refund, and dispute notifications.
6. Added deterministic retry and webhook deduplication through user-scoped keys.
7. Added startup and six-hour retention pruning for the approved 90-day lifecycle.
8. Added database, authorization, preference, isolation, pagination, read-state, retention, domain-event, and OpenAPI regression coverage.

Completion Criteria

1. Users can manage notifications without seeing another user or tenant's data.
2. Preferences are enforced at notification creation time.

Phase 13: File Storage Backend

Status: Complete

Objective: Support secure organization-scoped files through a replaceable provider boundary.

Decision Gate

1. Select AWS S3 or Cloudflare R2 as the reference provider.
2. Approve size limits and accepted content types.
3. Approve retention, deletion, malware, and orphan-cleanup policies.

Ordered Steps

1. Add file metadata, ownership, organization, object-key, type, size, and lifecycle fields.
2. Add provider-neutral upload, download, sign, and delete interfaces.
3. Implement the selected provider adapter.
4. Keep objects private and use short-lived signed URLs.
5. Enforce tenant capability checks before every file operation.
6. Validate declared and detected file types and sizes.
7. Clean up partial uploads, failed database writes, and orphaned objects.
8. Add provider-contract, authorization, and cleanup tests.

Completion Criteria

1. Authorized users can upload, retrieve, and delete tenant-scoped files.
2. Private objects are never exposed through permanent public URLs.
3. Database and object-store failure paths are recoverable.

Phase 14: Backend Production Hardening

Status: In Progress

Objective: Meet measurable backend security, reliability, deployment, and operational criteria.

Ordered Steps

1. Add structured production logging and sensitive-field redaction.
2. Select and integrate error monitoring and alerting.
3. Add security headers, proxy trust rules, request-size limits, and timeouts.
4. Replace in-memory rate limits if horizontal deployment is approved.
5. Review secrets, cookies, JWTs, OAuth, CORS, CSRF, and provider callbacks.
6. Review tenant scoping, transaction boundaries, indexes, and slow queries.
7. Add production Docker images, non-root runtime, health checks, and graceful shutdown.
8. Document deployment migrations, rollback, backups, restore, and incidents.
9. Run dependency, API security, and supported-platform reviews.
10. Define measurable production acceptance criteria and record results.

Provider-Independent Baseline Completed

1. Added structured JSON logging with sensitive-field redaction.
2. Added security headers, explicit body limits, proxy-hop configuration, HTTP timeouts, and configurable in-memory throttling.
3. Added PostgreSQL statement timeouts and parameter-free slow-query warnings.
4. Added a multi-stage production image, non-root runtime, migration target, and container health check.
5. Added deployment, rollback, backup, restore, incident, and acceptance procedures.
6. Added HTTP-security, logging-redaction, timeout, and runtime-configuration tests.

Explicit Deferrals

1. Error monitoring, alert delivery, centralized log retention, and uptime monitoring await provider selection.
2. Automated off-site backups await the production PostgreSQL and storage topology.
3. Distributed rate limiting remains deferred until horizontal deployment is approved.

Completion Criteria

1. The backend can be deployed, monitored, backed up, restored, and rolled back.
2. Security and reliability checks have documented evidence.

Phase 15: Backend CI/CD, Testing, and Documentation

Status: Pending

Objective: Make the backend repeatable, verifiable, and maintainable by buyers and contributors.

Ordered Steps

1. Add GitHub Actions for frozen install, format check, lint, unit tests, e2e tests, Prisma validation, migration checks, audit, and build.
2. Provision an isolated PostgreSQL service for CI integration tests.
3. Add regression suites for authentication, OAuth linking, tenant isolation, RBAC, webhooks, and entitlements.
4. Publish versioned Swagger/OpenAPI artifacts.
5. Decide whether to publish a generated frontend client or shared schema package.
6. Document setup, environments, migrations, providers, deployment, extension, upgrades, and troubleshooting.
7. Define semantic versioning, tags, changelog, compatibility, and support policies.
8. Add dependency-update automation with required verification.

Completion Criteria

1. Every pull request and release runs the required backend checks automatically.
2. A new buyer can configure, test, deploy, and extend the backend from documentation.

Release: Backend Support for Beta and Commercial Launch

Status: Pending

Objective: Supply a stable backend artifact and operational support for the ShipFlow v1 release.

Ordered Steps

1. Freeze the release API and database migration baseline.
2. Create realistic demo seed data without production credentials.
3. Support beta setup, issue triage, and release-blocking fixes.
4. Publish versioned release notes and migration guidance.
5. Define supported Node, pnpm, PostgreSQL, browser-client, and provider versions.
6. Prepare source delivery, update delivery, repository access, and support procedures.

Completion Criteria

1. The release tag is reproducible from a clean checkout.
2. Upgrade and rollback paths are documented.
3. No release-blocking backend defects remain.

Backend-to-Frontend Contract

1. Swagger/OpenAPI is the authoritative initial HTTP contract.
2. All endpoints remain under `/api/v1` until a versioning decision changes it.
3. The frontend stores access tokens in memory and sends refresh requests with credentials enabled.
4. Refresh tokens remain inaccessible to frontend JavaScript.
5. The frontend must handle the standard API error response and request ID.
6. Tenant-scoped routes use explicit organization IDs.
7. The backend remains authoritative for membership, permissions, entitlements, and tenant isolation.
8. Breaking API changes require coordinated release notes and frontend compatibility review.

Immediate Next Backend Task

Complete the remaining Phase 14 provider decisions: deployment target, managed PostgreSQL topology, centralized logging and error monitoring, off-site backup automation, and whether the first production release needs horizontal scaling.